

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..27

ROUTING

 Copy as Markdown

Routing

Routing decides which of your `Route` classes answers an incoming HTTP request. This page covers the matching rules, HTTP method filtering, path parameters, wildcards, fallback handling, and virtual hosts.

Route classes

The `Route` base class gives you complete control over request handling. By extending `Route` and implementing `handleCall()`, you can build REST APIs, serve files, or handle any custom HTTP interaction. This is ideal when you need to work directly with request bodies, headers, and response formats.

```
class ApiRoute extends Route {
  ApiRoute() : super(methods: {Method.get, Method.post});

  @override
  Future<Result> handleCall(Session session, Request request) async {
    // Access request method
    if (request.method == Method.post) {
      // Read request body
      final body = await request.readAsString();
      final data = jsonDecode(body);

      // Process and return JSON response
      return Response.ok(
        body: Body.fromString(
          jsonEncode({'status': 'success', 'data': data}),
          mimeType:MimeType.json,
        ),
      );
    }

    // Return data for GET requests
    return Response.ok(
      body: Body.fromString(
        jsonEncode({'message': 'Hello from API'}),
        mimeType: MimeType.json,
      ),
    );
  }
}
```



```
    };  
  }  
}
```

You need to register your custom routes with the built-in router under a given path:

```
// Register the route  
pod.webServer.addRoute(ApiRoute(), '/api/data');
```

INFO

The examples in this documentation omit error handling for brevity.

How requests are matched

Requests are matched by specificity, not by the order routes were registered. A literal path segment such as `/users/list` wins over a dynamic one such as `/users/:id` or a wildcard. Registering two routes with the same method and path throws at startup with `Invalid argument(s): Conflicting values`, so conflicts surface immediately rather than silently shadowing each other.

HTTP methods

Routes answer GET requests only by default. Use the `methods` parameter to specify which HTTP methods a route responds to:

```
class UserRoute extends Route {  
  UserRoute() : super(  
    methods: {Method.get, Method.post, Method.delete},  
  );  
  // ...  
}
```

A request whose method is not in the set gets an automatic `405 Method Not Allowed` response with an `Allow` header listing the accepted methods.

Path parameters

Define path parameters in your route pattern using the `:paramName` syntax:

```
pod.webServer.addRoute(UserRoute(), '/api/users/:id');  
// Matches: /api/users/123, /api/users/456, etc.
```

You can use multiple path parameters in a single route:

```
pod.webServer.addRoute(route,('/:userId/posts/:postId'));  
// Matches: /123/posts/456
```

Wildcards

Routes also support wildcard matching for catching all paths:

```
// Single-level wildcard - matches /item/foo but not /item/foo/bar  
pod.webServer.addRoute(ItemRoute(), '/item/*');  
  
// Tail-match wildcard - matches /item/foo and /item/foo/bar/baz  
pod.webServer.addRoute(ItemRoute(), '/item/**');
```

❗ TAIL MATCHES

The `/**` wildcard is a **tail-match** pattern and can only appear at the end of a route path (e.g., `/static/**`). Patterns like `/a/**/b` are not supported.

Access the matched path information through the `Request` object:

```
@override  
Future<Result> handleCall(Session session, Request request) async {  
  // Get the remaining path after the route prefix  
  final remainingPath = request.remainingPath;  
  
  // Access query parameters  
  final query = request.queryParameters.raw['query'];  
  
  return Response.ok(  
    body: Body.fromString('Path: $remainingPath, Query: $query'),  
  );  
}
```

Fallback routes

You can set a fallback route that handles requests when no other route matches:

```
class NotFoundRoute extends Route {  
  @override  
  Future<Result> handleCall(Session session, Request request) async {  
    return Response.notFound(  
      body: Body.fromString('Page not found: ${request.url.path}'),  
    );  
}
```

```

    }
}

// Set as fallback
pod.webServer.fallbackRoute = NotFoundRoute();

```

The fallback runs only when no route matches the request. If a route matches but its handler returns a 404, the fallback is not invoked. To rewrite those responses, use [FallbackMiddleware](#) instead.

🔗 ADVANCED: GROUPING ROUTES INTO MODULES

As your web server grows, managing dozens of individual route registrations can become unwieldy. You can group related endpoints into reusable modules by overriding the `injectIn()` method. This lets you register multiple handler functions instead of implementing a single `handleCall()` method.

Here's an example:

```

class UserCrudModule extends Route {
  @override
  void injectIn(RelicRouter router) {
    // Register multiple routes with path parameters
    router
      ..get('/', _list)
      ..get('/:id', _get);
  }

  // Not used when injectIn registers its own handlers.
  @override
  Future<Result> handleCall(Session session, Request request) =>
    throw UnimplementedError();

  // Handler methods
  Future<Result> _list(Request request) async {
    final session = await request.session;
    final users = await User.db.find(session);

    return Response.ok(
      body: Body.fromString(
        jsonEncode(users.map((u) => u.toJson()).toList()),
        mimeType: MimeType.json,
      ),
    );
  }

  static const _idParam = IntPathParam(#id);
  Future<Result> _get(Request request) async {
    int userId = request.pathParameters.get(_idParam);
    final session = await request.session;
    final user = await User.db.findById(session, userId);

    if (user == null) {
      return Response.notFound(
        body: Body.fromString('User not found'),

```

```

    });
}

return Response.ok(
    body: Body.fromString(
        jsonEncode(user.toJson()),
        mimeType: MediaType.json,
    ),
);
}
}

// Register the entire CRUD module under /api/users
pod.webServer.addRoute(UserCrudModule(), '/api/users');

```

This creates `GET /api/users` and `GET /api/users/:id` routes.

Handlers receive only a `Request` parameter. To access the `Session`, use `request.session`, which returns a `Future<Session>` that must be awaited. The `handleCall()` override is still required by the base class, but it is never called when `injectIn` registers its own handlers, so throwing `UnimplementedError` is the expected pattern. Typed path parameter accessors such as `IntPathParam` are covered in [Request data](#).

Virtual host routing

Virtual host routing allows you to serve different content based on the `Host` header of incoming requests. This is useful for multi-tenant applications, or for serving different front-ends from distinct subdomains with a single server instance, such as an API on `api.example.com`, a web app on `www.example.com`, and an admin panel on `admin.example.com`.

How it works

By default, routes match requests from any host.

```

// Route that responds to any host (default behavior)
class PublicRoute extends Route {
    PublicRoute() : super(); // host defaults to null

    @override
    Future<Result> handleCall(Session session, Request request) async {
        return Response.ok(
            body: Body.fromString('Public endpoint'),
        );
    }
}

```

You can restrict a route to a specific host by providing the `host` parameter when creating the route:

```
// Route that only responds to api.example.com
class ApiRoute extends Route {
  ApiRoute() : super(host: 'api.example.com');

  @override
  Future<Result> handleCall(Session session, Request request) async {
    return Response.ok(
      body: Body.fromString(
        jsonEncode({'message': 'API endpoint'}),
        mimeType: MimeType.json,
      ),
    );
  }
}
```

Registering host-specific routes

Register routes with host restrictions just like regular routes:

```
// Route that only responds to api.example.com
pod.webServer.addRoute(ApiRoute(), '/v1');

// Route that only responds to www.example.com
pod.webServer.addRoute(
  SpaRoute(
    Directory('web/app'),
    fallback: File('web/app/index.html'),
    host: 'www.example.com',
  ),
  '/',
);

// Route that responds to any host (default behavior)
pod.webServer.addRoute(HealthRoute(), '/health');
```

Supported route types

All route types support virtual host routing:

- **Custom routes** - Pass `host` to the `Route` constructor
- **StaticRoute** - Use `host` parameter in `StaticRoute.directory()` or `StaticRoute.file()`
- **SpaRoute** - Use `host` parameter in the `SpaRoute` constructor
- **FlutterRoute** - Use `host` parameter in the `FlutterRoute` constructor

```
// Static files for a specific host
pod.webServer.addRoute(
  StaticRoute.directory(
    Directory('web/admin'),
    host: 'admin.example.com',
  ),
);
```

```
    '/static/',  
  );  
  
  // SPA for a specific host  
  pod.webServer.addRoute(  
    SpaRoute(  
      Directory('web/app'),  
      fallback: File('web/app/index.html'),  
      host: 'app.example.com',  
    ),  
    '/',  
  );
```

Next steps

- **Request data** - Access path parameters, query parameters, headers, and body
- **Middleware** - Intercept and transform requests and responses
- **Static files** - Serve static assets
- **Server-side HTML** - Render HTML on the server